J Intell Manuf (2016) 27:131–148
DOI 10.1007/s10845-013-0818-y

A self-adaptation scheme for workflow management in multi-agent systems

Fu-Shiung Hsieh · Jim-Bon Lin

Received: 27 February 2013 / Accepted: 23 July 2013 / Published online: 6 August 2013

Abstract Business processes, operational environment, variability of resources and user needs may change from time to time. An effective workflow management software system must be able to accommodate these changes. The ability to dynamically adapt to changes is a key success factor for workflow management systems. Holonic multi-agent systems (HMS) provide a flexible and reconfigurable architecture to accommodate changes based on dynamic organization and collaboration of autonomous agents. Although HMS provides a potential architecture to accommodate changes, the dynamic organization formed in HMS poses a challenge in the development of a new software development methodology to dynamically compose the services and adapt to changes as needed. This motivates us to study and propose a methodology to design self-adaptive software systems based on the HMS architecture. In this paper, we formulate a workflow adaptation problem (WAP) and propose an interaction mechanism based on contract net protocol (CNP) to find a solution to WAP to compose the services based on HMS. The interaction mechanism relies on a service publication and discovery scheme to find a set of task agents and a set of actor agents to compose the required services in HMS. We propose a viable self-adaptation scheme to reconfigure the agents and the composed services based on cooperation of agents in HMS to accommodate the changes in workflow and capabilities of actors. We propose architecture for our design methodology and present an application scenario to illustrate our idea.

F.-S. Hsieh (✉) · J.-B. Lin
Department of Computer Science and Information Engineering, Chaoyang University of Technology, Taichung, Taiwan
e-mail: fshsieh@cyut.edu.tw

Keywords Workflow · Holonic system · Multi-agent system · Self-adaptation · Petri nets

Introduction

The business world has been facing major challenges as it undergoes sophisticated and frequently changing demands from customers, global competition and technological advances. To address these challenges, enterprises need to change the way they do business and adopt innovative organizational forms, technologies and solutions to increase their responsiveness and efficiency. The next generation advanced manufacturing technologies will rely on cooperation and collaboration of business partners to share costs, risks and expertise as no single company has all the expertise needed. Traditional centralized hierarchical organizations cannot effectively respond to rapidly changing demands, innovative production processes and highly dynamic business partnership in the supply chain because they do not fully exploit the benefits offered by advanced information and communication technologies. How to take advantage of the advancement of technologies to effectively support the operation and acquire competitive advantage is critical for enterprises to survive. To reap the potential technological benefits, new organizational structure and strategy must be developed to effectively manage the business processes/workflows, resources and changes in business environment to support inter-enterprise collaboration.

Motivated by these needs, we concentrate on significant research issues in this paper. Our objectives are to study (1) how to effectively organize manufacturing resources to achieve a business goal based on the combination of communication, coordination and cooperation support, (2) how

to construct the required workflow management architecture to achieve optimization of manufacturing organization, (3) how to dynamically reconfigure workflow to accommodate changes, (4) how to develop an effective methodology to support adaptation of systems in changing business environment and (5) how to provide the required information to the users at point of need in changing networked organization.

A workflow defines a set of activities and a set of procedural rules that determine the specific order in which these activities must be executed to achieve a business goal. Typical workflows in an enterprise are subject to change due to uncertainties in business processes, operational environment, variability of resources and people involved, etc. An effective workflow management software system must be able to accommodate these changes. Therefore, a challenging issue in the context of business process management is the ability to deal with process changes (van der Aalst and Jablonski 2000). The ability to dynamically self-adapt to changing environments, resource variability, changing user needs, and system faults is a key factor for workflow software systems. Traditional approaches to self-adaptation are hard-wired within each application, which are often highly application-specific, static in nature, and tightly bound to the code, and can not effectively deal with those complex environments. Architecture-based adaptation frameworks are proposed in (Oreizy et al. 1999; Garlan et al. 2004) to handle cross system adaptation. A well-accepted design principle in architecture-based management includes using component based technologies to develop management system and application structure (Kon et al. 2005). The current trend on component model regards components as run-time manageable entities instead of design-time artifacts (Coulson et al. 2008). However, such approach maintains little design-time knowledge on the run-time entities and makes it hard to achieve accurate and correct run-time adaptation.

Process changes may have impact on a single running instance or all instances that will run in the near future. The latter are called evolutionary changes (van der Aalst and Basten 2002), which may be due to adoption of a new business strategy. van der Aalst and Jablonski propose four approaches to workflow evolution (van der Aalst and Jablonski 2000). These approaches include Forward recovery (aborting existing cases), Backward recovery (stopping existing workflow and restarting according to the new one), Proceed (handling existing cases in the old way while handling new cases in the new way) and Transfer (transferring existing cases to the new workflow definition). The capability to dynamically adapt in-progress workflows is an essential requirement for any workflow management system. Rinderle et al. systematically classifies different approaches and discusses their strengths and limitations related to dynamic workflow changes (Rinderle et al. 2004). Recently, Cicirelli et al. (2010) propose an approach to decentralized enactment and dynamic reconfiguration of business processes based on a service-oriented architecture. Despite the above mentioned results, there is still a lack of study on self-adaptation and accommodation of process changes.

The objectives of this paper are to propose a viable software design methodology to achieve autonomic self-adaptation in workflow management systems. To achieve architecture-based self-adaptation, flexible system architecture must be adopted. Holonic multi-agent systems (HMS) provide a flexible and reconfigurable architecture to deal with changes based on dynamic organization and collaboration of autonomous agents (Koestler 1967). An agent is an autonomous, co-operative and intelligent entity able to collaborate with other agents to process the tasks (Ferber and Gutknecht 1998; Wooldridge 1999). In HMS, a set of agents that autonomously cooperate to achieve a goal forms a holarchy. The loosely coupled structure of agents makes it possible to attain for the benefits of stability in the face of disturbances, adaptability and flexibility in the face of change, and efficient use of available resources. In existing literature, the holonic concept has been recognized as a paradigm to achieve agility, accommodate changes and meet customers' requirements flexibly in manufacturing systems (Balasubramanian et al. 2001; Wyns 1999; Christensen 1994; Hsieh 2008, 2009; Hsieh and Chiang 2011; Hsieh and Lin 2012). Although HMS provides a potential architecture to accommodate changes, the dynamic organization formed in HMS poses a challenge in the development of a new software design methodology to dynamically compose the services and adapt to changes as needed (Cicirelli et al. 2010; Leitão and Restivo 2006). This motivates us to study and propose a methodology to design and implement self-adaptive workflow systems based on the HMS architecture.

In this paper, we formulate a workflow adaptation problem (WAP) and propose an interaction mechanism based on contract net protocol (CNP) (Smith 1980) to find the solution to WAP to compose the services based on HMS. The interaction mechanism relies on a service publication and discovery scheme to find a set of task agents and a set of actor agents to form a holarchy to compose the required services in HMS. We propose a self-adaptation scheme to reconfigure the holarchy and compose services to respond to changes in processes.

To dynamically compose workflow services based on HMS architecture, we model the service provided by software as an agent instead of a static passive component. In our approach, the service provided by each agent is modeled as an atomic service that can be combined with the atomic services provided by other agents to form a full fledge service to perform tasks. The atomic services can be classified into two categories: atomic tasks and atomic

activities. An atomic task specifies the set of operations and the precedence constraints of the operations in a task. An atomic activity captures the capabilities or functions of an actor. A task agent hosts an atomic service that can be combined with the atomic services provided by other task agents dynamically to compose the desired services and achieve the goal.

A critical factor in the development of a self-adaptive workflow management system is interoperability issue. In real business environment, the workflow management systems used by enterprises are heterogeneous. Application of a self-adaptation scheme to reconfigure the workflows relies on effective architecture to address the heterogeneity problems. Chen et al. (2008) reviewed architectures for enterprise integration, recent developments on architectures for enterprise interoperability and discusses future trends. The concept of integrated enterprise and the implementation and interoperability of networked enterprises are a key concept to face new challenges in manufacturing sector. Panetto and Molina (2008) describe challenges, trends and issues that must be addressed to support the generation of new technological solutions. There are several existing reference models or frameworks to attain interoperability at the enterprise level: LISI reference model (C4ISR 1998), IDEAS interoperability framework (IDEAS 2002), ATHENA interoperability framework (ATHENA 2003) and Enterprise interoperability framework developed within INTEROP NoE (Chen et al. 2005). In addition to the interoperability issue, another issue is to apply a software engineering methodology that supports the development of multi-agent system application. Several agent-oriented software engineering (AOSE) methodologies with diversity in scope, focus and approach have been proposed, e.g., MaSE (DeLoach and Kumar 2005), GAIA (Wooldridge et al. 2005), PROMETHEUS (Padgham and Winikoff 2005), TROPOS (Bresciani et al. 2004) and MOBMAS (Tran and Low 2008). Wherever possible, our proposed methodology reuses and/or enhances techniques and model definitions from existing AOSE methodologies to take advantage of their strengths.

To facilitate service composition, a modeling tool or specification language is required to represent atomic services. The modeling tool must meet two requirements. First, the model must have well-established formalism for modeling, analysis and verification of the properties of composed services. Second, the model must be accompanied with a standardized interchange format so that composition of services can be performed efficiently. In existing literature, many workflow specification languages have been proposed, e.g. XPDL (Workflow Management Coalition 1998), Business Process Model and Notation (BPMN) (Object Management Group 2009) and Web Services Business Process Execution Language (WS-BPEL) (OASIS 2009). However, these workflow specification languages lack formal analysis method. Petri net (Murata 1989) is a model that meets these two requirements. Application of Petri nets in workflow management have been studied by van der Aalst (1998). The Petri Net Markup Language (PNML) (Weber and Kindler 2002) is an XML-based interchange format for Petri nets. Many Petri net tools support PNML format, e.g. WoPeD, Renew, PNK, PEP, VIPtool) (Billington et al. 2003). Therefore, we adopt Petri net to model the services provided by agents and use PNML format to represent the model.

Our design methodology is broken down into five parts. First, we construct task models and actor activity models in Petri nets using PNML editors. Second, we propose a service publication and discovery scheme based on Petri net models. Third, an interaction mechanism based on CNP is proposed to form a holarchy that consists of the best agents and combine the PNML files of the task models and actor activity models of agents into a single PNML file that represents the complete Petri net model of the composed service. Fourth, we propose an architecture to facilitate generation of the list of actions for the actor agents in a given system state. Fifth, we propose a self-adaptation scheme to respond to the process changes.

In workflow systems, changes in processes can be classified into structural changes and non-structural changes. Structural changes refer to addition, modification and removal of some part of a workflow. Non-structural changes include changes in processing time, number of available resources and available time slots of resources. An effective reconfiguration scheme needs to be able to deal with these changes. To cope with changes in a workflow system, we propose a self-adaptation scheme to re-structure the holarchy to reflect and respond to the changes. Finding a solution from scratch from the point a change occurs is not an appropriate approach as the solution may lead to chaos. An effective approach to reconfiguration must be based on the nominal solution so that the impact on the operation of the existing agents can be significantly reduced. To achieve effective self-adaptation, a viable solution is based on cooperation of agents in HMS to accommodate the changes in workflows and capabilities of actor agents.

The remainder of this paper is organized as follows. In "Workflow adaptation problem" section, we introduce the workflow adaptation problem based on HMS architecture. We review existing framework/technologies in "A survey of existing framework/technologies" section. We propose a development methodology in "Software development" section. In "Internal design of agents" section, we detail the internal design of agents, including workflow models for task agents and activity models for actors. In "Adaptation of workflows" section, we propose a self-adaptation scheme to reconfigure workflows in HMS. We present architecture

for the implementation of software system and an application scenario to demonstrate our developed software tool in "Application scenario" section. We conclude and compare this paper with existing studies in "Conclusion" section.

Workflow adaptation problem

Workflow management is concerned with allocation of resources to achieve a certain business objective in a timely and efficient manner. Typically, a variety of actors are involved in the business processes to accomplish a task. Figure 1a shows different steps for handling an order, from sale, costing, offering, counter offering, manufacturing, quality assurance, packaging to shipping.

Each step requires distinct resources for processing. For example, in sale step, sale staffs are involved in the negotiation as well as contracting. Costing of an order requires the assistance from the accountants. Therefore, a message is forwarded to the accountants for costing. Once the order contract is established, it will be released to the shop floor for production. In the production process, different workers are assigned to perform the relevant operations. Each final product is then forwarded to the quality control staff for verification. Once the required qualified products are available, the workers package and then ship the products to the customer. The above scenario exhibits very complicated process flows with intensive interactions between different workers. How to effectively organize the concurrent workflows, allocate operations to different actors, dynamically generate the user interface to guide the actors to take the next action and respond to process changes is a challenge.

HMS provides a flexible architecture to deal with changes based on dynamic organization and collaboration of autonomous agents. To solve the aforementioned problem based on HMS, a brief introduction to HMS is given first. The dynamics and behaviors of HMS are determined by the interactions of agents. For a workflow system, there are two types of agents: task agents and actor agents. A task agent represents a task to be processed. An actor agent is the abstraction of a worker or a customer in the system. To process a task relies on the cooperation of a set of task agents and actor agents.

For the workflow system in Fig. 1a, the different actor agents involved in processing an order are classified into order manager, accounting/costing staff, production manager, quality assurance staff and packaging/shipping manager. For example, the actor agent involved in order processing is an order manager and the actor agent whose main ownership is packaging/shipping the products and delivering them to the customer is a packaging/shipping staff. Figure 1b shows our proposed architecture for workflow management based on HMS architecture. For the example of Fig. 1a, the workflows of the task agents are defined in Table 1 and the roles of different types of actor agents are defined in Table 2.

Individual actor agents and task agents cannot process a complex task alone. In HMS, a set of agents that can autonomously cooperate to achieve a goal forms a holarchy. Figure 2 shows a holachy for the example of Fig. 1a. The holarchy in Fig. 2 consists of task agents $h_1, h_2, h_3, h_4, h_5, h_6$ and actor agents $a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$. In our proposed methodology, process/workflow models are used to capture the workflows of task agents. The actor activity models are used to capture the activities of actor agents.

Workflows in an enterprise are subject to change due to changing business processes, operational environment, resources and user needs. An effective workflow management software system must be able to accommodate these changes. The ability to dynamically self-adapt to these changes is a key success factor for workflow management systems. The key issues include the development of an effective method to dynamically form a holarchy that composes the required services for performing a task in workflow systems and accommodate changes in the system.

The problem to be studied in this paper is to compose the services required and adapt to changes as needed to achieve a goal based on HMS architecture. In workflow systems, changes in processes can be classified into structural changes and non-structural changes. Structural changes refer to addition, modification and removal of some parts of a workflow. Non-structural changes include changes in processing time, number of available resources and available time slots of resources. In this paper, we will propose a self-adaptation scheme to deal with structural changes and non-structural changes in the system. To formulate the problem, let $\boldsymbol{A}$ denote the set of actor agents and $\boldsymbol{N}$ denote the set of task agents. A complete workflow is a holarchy denoted by $H(A, N)$, where N denotes the set of task agents in H and A denotes the set of actor agents that take part in the activities in H. Let Δ denote the changes in the system. The workflow adaptation problem (WAP) aims to find a new holarchy $H(A', N')$ that satisfies the requirements based on the nominal holarchy $H(A, N)$ according to Δ.

In our holonic system approach, we assume each agent provides an atomic service. A holarchy is formed by composing the atomic services provided by the agents in the holarchy. To develop a viable solution methodology for WAP, five issues need to be addressed. First, a framework or infrastructure that supports interoperation of agents must be adopted. Second, a methodology to model the internal processes of agents is required. Third, a mechanism for discovery of the atomic services provided by agents is required. Fourth, an interaction protocol for agents to negotiate and coordinate with each other must be adopted. Fifth,

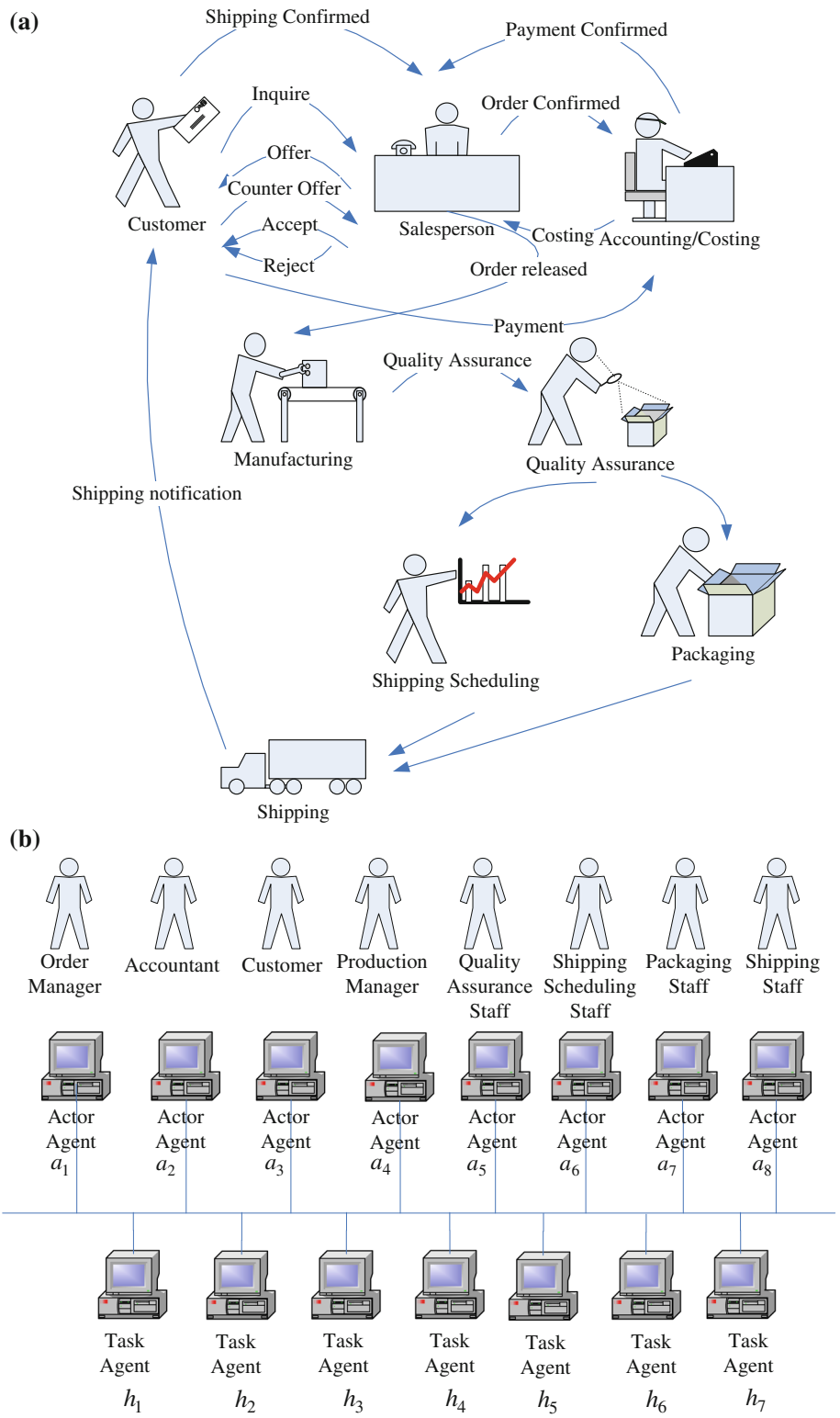

Fig. 1 **a** Task force and activities. **b** Architecture for workflow management

a workflow adaptation algorithm to analyze whether the constraints can be satisfied and determine the minimal cost solution. To solve the aforementioned problem, we first survey the existing frameworks/technologies that support the development of a self-adaptation scheme for workflow systems.

Table 1 Workflows of task agents

Task agents	Operations in the corresponding workflow
h_1	"inquire", "confirm", "costing", "offering", "counter offering" and "decision"
h_2	"reject" an order and "notify" the customer.
h_3	"accept" an order and "release" an order.
h_4	"payment" and "confirm"
h_5	"manufacture"
h_6	"quality assurance", "packaging", "shipping" the products and "confirming shipping"
h_7	"finalize" the order

Table 2 Actor agents

Actor agent	Role
a_1	Order manager
a_2	Accounting/costing staff
a_3	Customer
a_4	Production control staff
a_5	quality assurance staff
a_6	Shipping scheduling staff
a_7	Packaging staff
a_8	Shipping staff

A survey of existing framework/technologies

Our survey of the existing framework/technologies covers three subjects: frameworks that focus on interoperability, standards for specifying workflow/Processes and software engineering methodologies for MAS.

Interoperability framework

Interoperability is the ability for two systems to understand and use functionality of each other (Chen and Vernadat 2002, 2004). The word "interoperate" implies that one system performs an operation for another system. In the context of networked enterprises, interoperability refers to the ability of interactions between enterprise systems. Interoperability is considered as significant if the interactions can take place at least on three different levels: data, services and processes, with a semantics defined in a given business context.

LISI provides a maturity model and a process for determining joint interoperability needs, and selecting pragmatic solutions and a transition path for achieving higher states of capability and interoperability (C4ISR 1998). It covers four enabling attributes of interoperability known as PAID, namely: procedures, applications, infrastructure (hardware, communications, security and system services) and data.

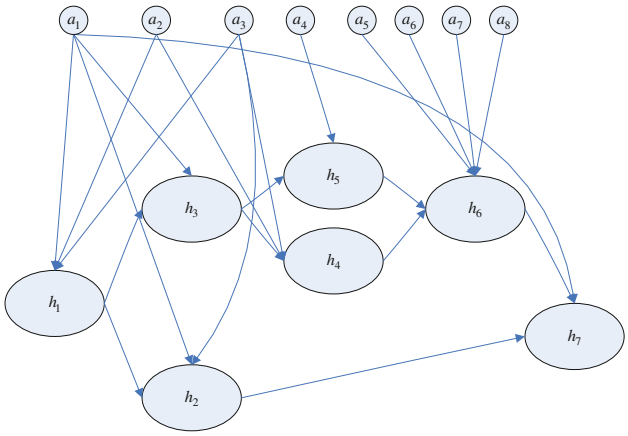

Fig. 2 A holarchy for Fig. 1

The IDEAS interoperability framework is carried out in Europe under FP5 (Fifth Framework Programme) to address enterprise and manufacturing interoperability issues. 'Interoperability is achieved on multiple levels: inter-enterprise coordination, business process integration, semantic application integration, syntactical application integration and physical integration" (IDEAS 2002). The ATHENA interoperability framework (AIF) (ATHENA 2003) is structured into three levels: (1) Conceptual level to identify research requirements and integrate research results of the ATHENA R&D projects, (2) Applicative level to integrate experience from R&D projects and technology testing in the pilot sites and (3) Technical level to conduct technology testing based on profiles and integrate prototypes of R&D projects. The INTEROP Network of Excellence (INTEROP NoE) framework (Chen et al. 2005) (i) defines the domain of enterprise interoperability through the elaboration of an interoperability framework by identifying three categories of barriers (conceptual, technological and organizational); (ii) identifies and structures the knowledge of the domain using the framework. It considers interoperations that take place at the various enterprise levels, including interoperability of data, services, processes and business. Coordination and interoperability are important aspects in today's global competing business environment. In the European project SUddEN, an approach that supports collaborative performance measurement system development to achieve coordination and organizational interoperability in supply networks is proposed (Weichhart et al. 2010). It allows partners to adapt their individual processes to improve organizational interoperability.

Seamless interoperability among heterogeneous organizations becomes a critical issue. Chituc et al. (2008) presented a review of the most relevant approaches to achieving seamless interoperability in a collaborative networked environment. A two-layered operational infrastructure aiming at attaining seamless interoperability in the footwear industry is introduced. A conceptual framework towards seamless

interoperability in collaborative networks is described in Chituc et al. (2009). It comprises six elements: (1) a messaging service; (2) a collaboration profile/agreement definition and management service; (3) five clusters of collaborative activities; (4) a centralized repository; (5) a set of business documents/contracting documents; (6) a performance assessment service. The framework is illustrated with a real implementation case of the footwear industry.

Interoperability has been mainly approached from an IT point of view or enterprise collaboration perspective. Naudet et al. (2010) proposed a formalization of interoperability grounded in the general system theory based on the Framework for Enterprise Interoperability (ISO 11354-1 2011). It provides a meta-model for ontological descriptions of systems, problems and solutions. A case example is presented to illustrate the proposed approach.

To reach interoperability, the system must eradicate heterogeneity. Blanc et al. addressed the problems of heterogeneity from two different points of view: semantic and organizational (Blanc et al. 2007). It presents an application of the proposed ECOGRAI method and implementation using a software tool in the frame of interoperability between manufacturing and maintenance companies.

Service-Oriented Architecture (SOA) is a paradigm for designing and developing software in the form of interoperable services. SOA (Huhns and Singh 2005; Ort 2005) is based on Web Services technologies, which can be loosely defined as self-describing software components accessible through the Internet. Web Services technology achieves interoperability between applications by using Web standards. Web services are essentially founded upon three major technologies to provide an industrial standard for deploying, publishing, discovering and invoking enterprise's services: Web Services Description Language (WSDL 2001), Universal Description, Discovery and Integration (UDDI 2007), and Simple Object Access Protocol (SOAP 2007). Web services have revolutionized the distributed computing paradigm and made various kinds of e-business such as inter-enterprise collaboration a reality. For example, several disparate departments within a company or different companies may develop and deploy SOA services in different implementation languages; their respective clients will benefit from a well-understood, well-defined interface to access them. Although SOA provides the framework for integrating cross-company services or technical interoperability, it does not address the semantic interoperability problem. A challenge is to develop SOA allowing interoperability of services 'on the fly' through dynamic accommodation and adaptation.

Existing standards for specifying workflow/processes

Workflow Reference Model (Workflow Management Coalition 1999) provides a general architectural framework that identifies interfaces and covers broadly five areas of functionality between a Workflow Management System (WfMS) and its environment: process definitions import and export; interaction with client applications and work-list handler software; software tools or applications invocation; interoperability between different WfMSs; administration and monitoring functions.

Business Process Model and Notation (OMG 2009) is a de facto standard of visual process notation from the OMG, endorsed by WfMC, and broadly adopted across the industry. But the BPMN standard defines only how the process definition is displayed on the screen. How to store and interchange those process definitions is outside the scope of the standard. XPDL bridges this gap by providing a file format that supports the BPMN process definition notation including graphical descriptions of the diagram and executable properties at run time. WS-BPEL and XPDL are different yet complimentary standards. WS-BPEL is an execution language to orchestrate web services. WS-BPEL does not define the graphical diagram, human oriented processes, subprocess, and many other aspects of a modern business process.

The ISO 18629 Process Specification Language (PSL) standard (ISO 18629-1 2004) aims at creating a neutral, high-level language for specifying processes and the interaction of multiple process-related applications throughout the manufacturing life cycle, including production scheduling, process planning, workflow management, business process reengineering, simulation, process realization, process modeling, and project management. It provides translation mechanisms between individual applications and PSL: e.g., syntactic translation, semantic translation to PSL, or from one application to another based on the PSL ontology.

However, these workflow specification languages lack formal analysis and verification method. Petri net (Murata 1989) is a model that has well-established formalism for modeling, analysis and verification with a standardized interchange format specified in ISO/IEC 15909-2 (2011). Application of Petri nets in workflow management have been studied by van der Aalst (1998). The Petri Net Markup Language (PNML) (Weber and Kindler 2002) is an XML-based interchange format for Petri nets. Many Petri net tools support PNML format, e.g. WoPeD, Renew, PNK, PEP, VIPtool) (Billington et al. 2003). Therefore, we adopt Petri net to model the services provided by agents and use PNML format to represent the model. Bresciani et al. proposed a decentralized Petri nets execution engine called PN-Engine that allows one to deploy and enact a new version of an existing process model without requiring the stopping/removal of older instances that are still running (Bresciani et al. 2004). The novel approach enables a decentralized migration procedure where concurrent portions of older

instances may migrate asynchronously to the new process model.

Software engineering methodologies for MAS

Several agent-oriented software engineering (AOSE) methodologies with diversity in scope, focus and approach have been proposed, e.g., MaSE (DeLoach and Kumar 2005), GAIA (Wooldridge et al. 2005), PROMETHEUS (Padgham and Winikoff 2005), TROPOS (Bresciani et al. 2004) and MOBMAS (Tran and Low 2008).

MAS-CommonKADS (Iglesias and Garijo 2005) defines a set of models (Agent Model, Task Model, Expertise Model, Coordination Model, Communication Model, Organization Model, and Design Model) that together provide a model of the problem to be solved.

The MaSE methodology (DeLoach and Kumar 2005) is a specialization of more traditional software engineering methodologies. The MaSE Analysis phase consists of three steps: Capturing Goals, Applying Use Cases, and Refining Roles. The Design phase has four steps: Creating Agent Classes, Constructing Conversations, Assembling Agent Classes, and System Design.

The Process for Agent Societies Specification and Implementation (PASSI) (Cossentino 2005) is a step-by-step requirement-to-code methodology for designing and developing multi-agent societies, integrating design models and concepts from both object-oriented (OO) software engineering and artificial intelligence approaches using the UML notation. The models and phases of PASSI encompass representation of system requirements, social viewpoint, solution architecture, code production and reuse, and deployment configuration supporting mobility of agents.

Due to the limited ontological support and the lack of consideration in interoperability with other systems in heterogeneous environments and design reuse in majority of existing AOSE methodologies, e.g., MAS-CommonKADS (Iglesias and Garijo 2005), MaSE (DeLoach and Kumar 2005), and PASSI (Cossentino 2005), MOBMAS (Tran and Low 2008) presents an ontology-based methodology for the analysis and design of multi-agent systems. The development process of MOBMAS consists of five activities: Analysis Activity, MAS Organization Design Activity, Agent Internal Design Activity, Agent Interaction Design Activity and Architecture Design Activity, which are highly iterative and incremental. Wherever possible, MOBMAS reuses and/or enhances techniques and model definitions from existing AOSE methodologies to combine the strengths of existing AOSE methodologies into it. Despite the advantages, the MOBMAS methodology still lacks a proper scheme to achieve adaptation of workflows. In this paper, we will propose a mechanism to facilitate workflow adaptation in MAS.

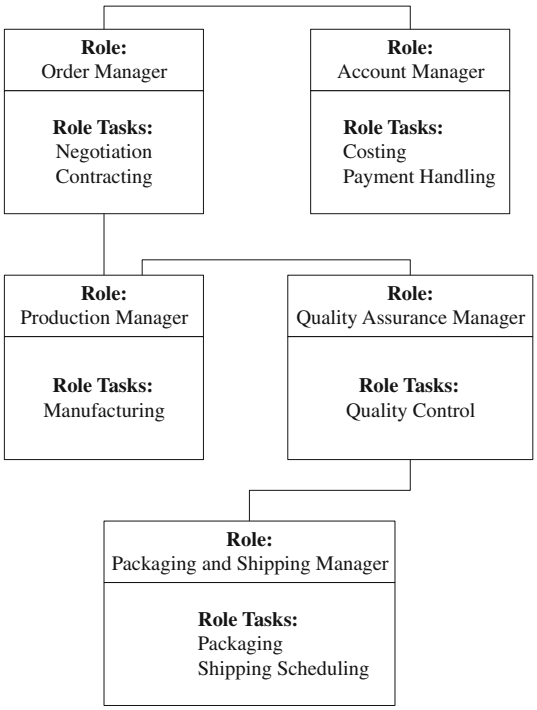

Fig. 3 Role diagram

Software development

In this section, we detail the software design of self-adaptation scheme for collaborative workflows. The key elements of software design consist of four parts: Role Diagram, Ontology, Agent Class Diagram, Agent Workflow Model, Agent Activity Model and Interaction Protocol diagram.

We identify a set of roles comprising the target MAS and the potential acquaintance between them. A role refers to a position in the MAS organization (Chen and Vernadat 2004). It serves as the building block for defining agent classes. The Role Diagram of our system is as follows. There are five types of roles in the system, including Order Manager, Account Manager, Production Manager, Quality Assurance Manager, Packaging and Shipping Manager. Each agent class will be associated with one or more roles. Therefore, roles also define the expected behavior of each agent class. Figure 3 shows the role diagram of workflow management systems.

Agent classes are built upon roles. We identify a set of agent classes comprising the target MAS by determining which roles are to be performed by which agent classes. All the agent classes in a system are described by Agent Class Diagrams which defines each agent class, including its dynamics, roles, belief conceptualization, goals and events. Figure 4 shows several examples of Agent Class Diagrams.

The internal design of each agent class includes the specification of each agent class's belief conceptualization, agent-goals, events and plans. Agent beliefs refer to the

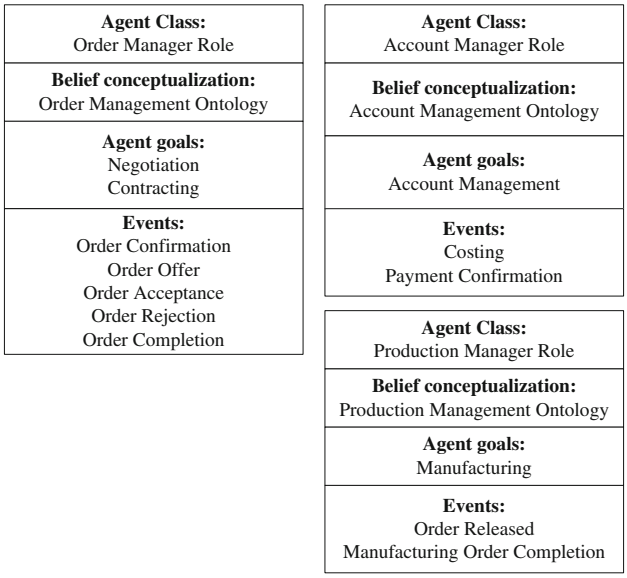

Fig. 4 Examples of agent class diagrams

informational state of an agent, in other words, its beliefs about the world. An agent-goal is a state of the world that an agent class would like to achieve or satisfy. An event is a significant occurrence in the environment which activates an agent to pursue its goals, or changes the agent's course of actions in achieving the goals. Plans of an agent are sequences of actions that the agent can perform to achieve one or more of its intentions. A plan may include the plans of other agents. To specify how each agent class behaves to achieve or satisfy each agent-goal, we consider the agent behavior of "planning" (Fikes and Nilsson 1971; Ambros-Ingerson and Steel 1988; Wood 1993; Bratman et al. 1988). Planning is a deliberative behavior that requires an agent to perform reasoning to dynamically choose among potential courses of actions for achieving an agent-goal, taking into account the current state of the environment and events. For planning of actions in a workflow system, the internal structure of an agent must be based on a proper workflow model.

Internal design of agents

There are two types of atomic services provided by agents in a workflow system. One is atomic task and the other is atomic activity. An atomic task specifies the operations and precedence constraints of the operations in a task. An atomic activity captures the capabilities or functions of an actor agent. An agent hosts an atomic service that can be combined with the atomic services provided by other agents to perform the tasks. To automate composition of services, a software model is required to represent atomic services. There are two requirements in the selection of modeling tools. First, the model must have a well-established formal mechanism for modeling, analysis as well as verification of the properties of composed services. Second, the model must be accompanied with a standardized interchange format so that composition of services can be performed efficiently. Petri net (Murata 1989) is a model that meets these two requirements. These advantages make Petri nets a suitable modeling and analysis tool. Furthermore, the emerging Petri Net Markup Language (PNML) (Weber and Kindler 2002) is an XML-based interchange format for Petri nets. PNML makes it possible for individual companies to exchange their process models in Petri nets. In this paper, PNML is used for sharing and exchanging the process models of agents. To model atomic tasks and atomic activities, we propose Petri net models to capture workflows as well as the resources required for performing the operations. A brief introduction to Petri net is first given.

Definition 5.1 A time Petri Net (PN) G is a five-tuple $G = (P, T, F, C, m_0)$, where P is a finite set of places, T is a finite set of transitions, $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation, $C : T \rightarrow R^+ \times (R^+ \cup \infty)$ is a mapping called static interval, which specifies the time interval(s) for firing each transition, where R^+ is the set of nonnegative real numbers, and $m_0 : P \rightarrow Z^{|P|}$ is the initial marking of the PN with Z as the set of nonnegative integers. A Petri net with initial marking m_0 is denoted by $G(m_0)$. A marking of G is a vector $m \in Z^{|P|}$ that indicates the number of tokens in each place under a state. ${}^\bullet t$ denotes the set of input places of transition t. A transition t is enabled and can be fired under m iff $m(p) \geq F(p, t) \forall p \in {}^\bullet t$.

In a time Petri net, each transition is associated with a time interval $[\alpha; \beta]$; where $\alpha$is called the static Earliest Firing Time, β is called the static Latest Firing Time, and $\alpha \leq \beta$; where α ($\alpha \geq 0$) is the minimal time that must elapse, starting from the time at which the transition is enabled until the transition can fire and $\beta (0 \leq \beta \leq \infty)$ represents the maximum time during which the transition is enabled without being fired. Firing a transition removes one token from each of its input places and adds one token to each of its output places. A marking m' is reachable from m if there exists a firing sequence s bringing m to m'. The reachability set $R(m_0)$ denotes all the markings reachable from m_0. A time Petri net $G = (P, T, F, C, m_0)$ is live if, no matter what marking has been reached from m_0, it is possible to ultimately fire any transition of G by progressing through some further firing sequence.

Definition 5.2 An atomic task provided by a task agent h_n is specified by a subclass of time Petri Net called acyclic marked graph $H_n = (P_n, T_n, F_n, C_n, m_{n0})$. As each transition represents a distinct operation in a task, $T_j \cap T_k = \Phi$ for $j \neq k$.

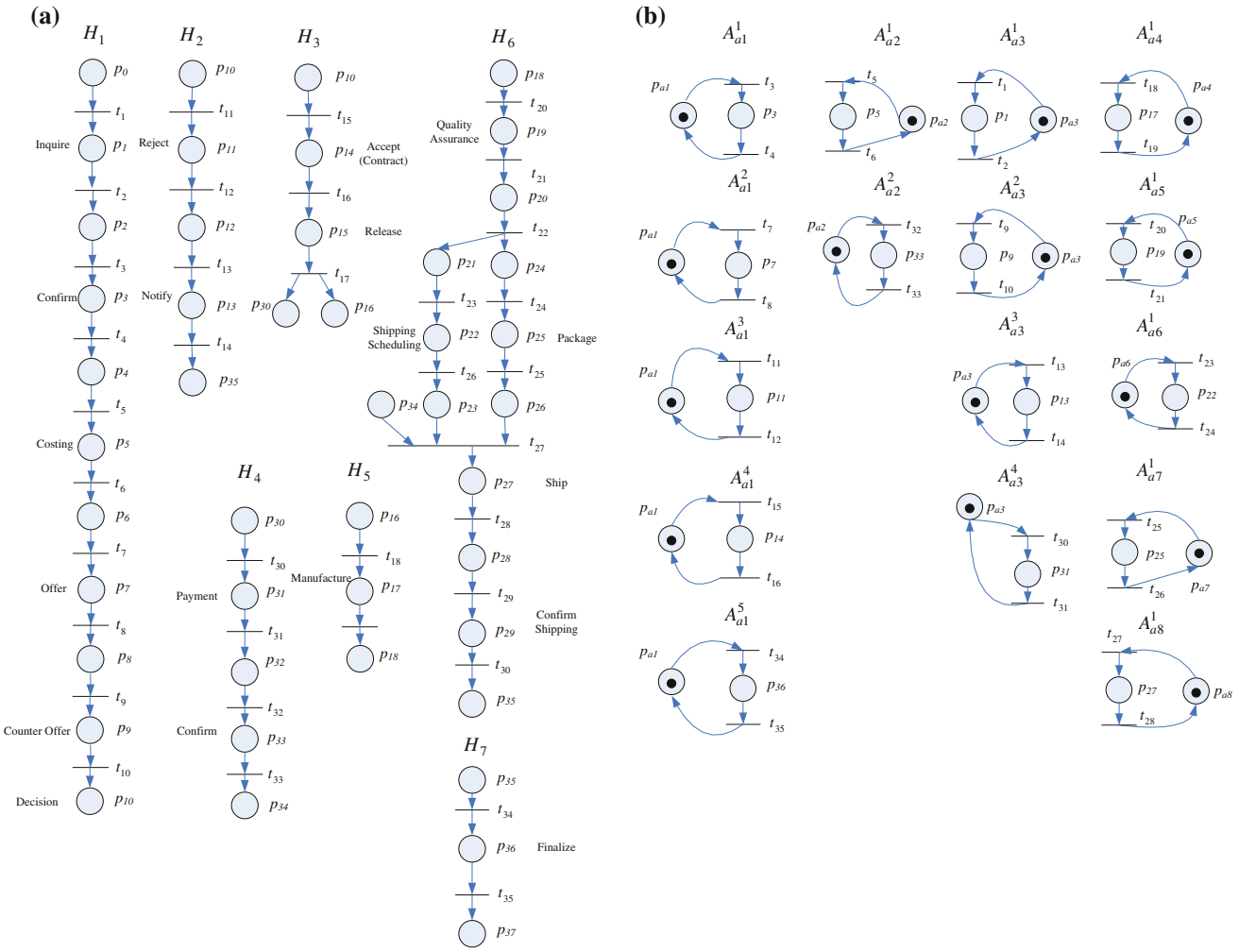

Fig. 5 **a** Workflows of task agents. **b** Activity models of actors a_1, a_2, a_3, a_4, a_5, a_6 and a_7

Example 1 Figure 5a illustrates the workflow Petri nets H_1, H_2, H_3, H_4, H_5, H_6 and H_7 for atomic tasks h_1, h_2, h_3, h_4, h_5, h_6 and h_7, respectively.

The atomic task model specifies a set of preconditions as well as a set of postconditions. Preconditions and postconditions of a task are represented by a set of terminal input places and a set of terminal output places defined as follows.

Definition 5.3 Given H_n, a place without input transition but with at least one output transition in H_n is called a terminal input place, which specifies a precondition of the workflow. A place without output transition but with at least one input transition in H_n is called a terminal output place, which specifies a postcondition of the workflow. We use ${}^\bullet h_n$ to denote the set of terminal input places of h_n whereas $h_n^\bullet$ to denote the set of terminal output places of h_n.

Example 2 For H_1, H_2, H_3, H_4, H_5, H_6 and H_7 in Fig. 5a, ${}^\bullet h_1 = \{p_0\}$, ${}^\bullet h_2 = \{p_{10}\}$, ${}^\bullet h_3 = \{p_{10}\}$, ${}^\bullet h_4 = \{p_{30}\}$, ${}^\bullet h_5 = \{p_{16}\}$, ${}^\bullet h_6 = \{p_{18}\}$ and ${}^\bullet h_7 = \{p_{35}\}$.

An activity describes the sequence of operations to be performed by an actor agent. The sequence of operations in an activity usually specifies some part of a task. Therefore, an activity captures the capabilities and functions of an actor agent. The set of actor agents to perform the operations of h_n is denoted by A_n. Let p_a denote the idle state place of actor agent a. The kth activity is described by a circuit place that starts and ends with an idle state place p_a. The circuit indicates allocation and de-allocation of the actor agent. The Petri net model for the kth activity of actor agent a, where $a \in A_n$, is described by a Petri net A_a^k as follows.

Definition 5.4 Petri net $A_a^k(m_a^k) = (P_a^k, T_a^k, F_a^k, C_a^k, m_a^k)$ denotes the kth activity for actor a, where $a \in A_n$. There is no common transition between A_a^k and $A_a^{k'}$ for $k \neq k\prime$. The initial marking $m_a^k(p_a) = 1$ and $m_a^k(p) = 0$ for each $P_a^k \backslash \{p_a\}$, where p_a is the resource idle state place.

Example 3 Figure 5b illustrates the actor agents' activities associated with the workflows in Fig. 5a.

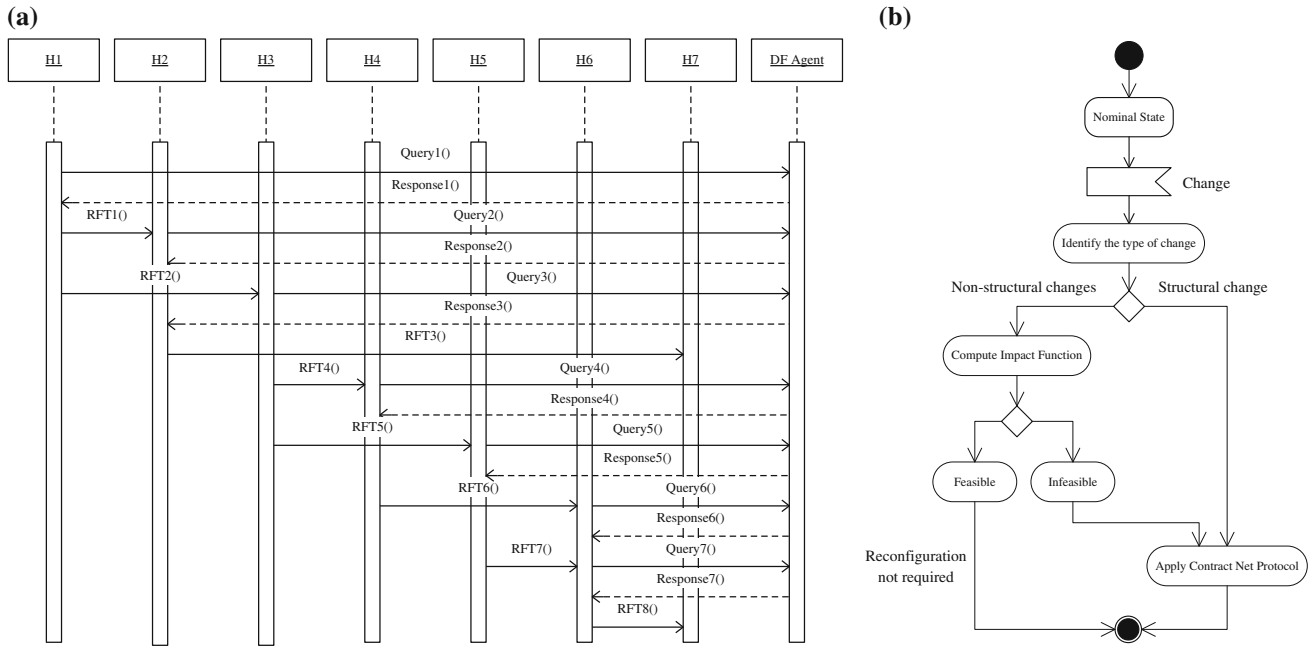

Fig. 6 **a** Discovery of services. **b** Self-adaptation of workflows

An operation t in $H_n = (P_n, T_n, F_n, C_n, m_{n0})$ performed by the kth activity of actor a is described by setting $C_n(t) = C_a^k(t)$.

Adaptation of workflows

Our approach to adaptation of workflows relies on an agent interaction mechanism to find a solution. Two basic mechanisms of agent interaction are direct interaction and indirect interaction. In direct interaction, agents exchange data by sending communication messages directly to each other following some interaction protocols. In indirect interaction, agents respond according to the signals or stimulus received from the environment but do not communicate with each other directly. In this paper, direct interaction is adopted. The interaction messages are typically expressed in agent communication languages (ACL) such as KQML (KQML Specification Document 1993) and FIPA-ACL (FIPA 2002a).

To solve WAP, we propose a solution methodology based on contract net protocol (CNP) and time Petri net models. Application of CNP requires an information infrastructure for one agent to discover services provided by the other agents. Many implementation framework, e.g. FIPA-based platforms such as JACK (A.O. Software 2012), Java Agent Development Environment (JADE) (jade.tilab.com), FIPA-OS (Emorphia 2003), and ZEUS (British Telecommunucations 2002), etc. support service publication and discovery infrastructure. With the service publication and discovery infrastructure, CNP can be applied to discover a set of agents to compose the required service. For example, JADE is a multi-agent platform that supports the development of multi-agent systems, publishing/discovery of agent services and contract net protocol. The JADE platform provides a built-in directory service to simplify publishing and discovery of services. An agent wishing to publish one or more services must provide the DF agent with a description including its AID and the list of published services.

To facilitate discovery of services, each task agent exposes its service as well as the corresponding terminal input places and terminal output places through the yellow page services provided by JADE. Our approach to making task agents discover other related agents is to publish the terminal input places and a set of terminal output places. A task agent h_n may connect to other task agents via a set of terminal output places or a set of terminal input places. A task agent H_m may connect to another task agent H_n only if $h_m^\bullet \cap {}^\bullet h_n \neq \Phi$.

To discover the services provided by other agents, an agent must search the yellow page services by providing the DF with a template description. The result of the search is the list of all the descriptions that match the provided template. A description matches the template if all the fields specified in the template are present in the description with the same values. Figure 6a shows how task agent h_1 searches the yellow page for other potential task agents.

Our self-adaptation scheme accommodates structural changes in workflows according to Fig. 6b. On detection

of a change in current process, an agent first identifies the type of change. If the change is a non-structural change, the impact function is computed to determine whether the current process is still feasible under the change or perturbation. If the current process is still feasible, reconfiguration is not required. Otherwise, the current process is infeasible. In this case, the reconfiguration process is initiated by applying CNP.

Whenever a change is detected, an agent will first compute the impact function and notify the downstream agent(s) the impact. On receiving the notification, each downstream agent then reacts similarly by compute the impact function based on its time Petri net model and notifies the downstream agent(s) the impact similarly. Eventually, the notification will reach all the downstream agents stemming from the first agent that detects the change. An agent that has received notification is called an affected agent. An affected agent without any downstream agent in the holarchy is called a terminal affected agent. Each terminal affected agent will evaluate with the overall impact information due to the change. The overall impact information will be sent back to its upstream affected agent by the terminal affected agent as needed. Depending on the impact of the change, the response of an affected agent varies. If the impact is acceptable for each affected agent, the existing workflow is still feasible and needs not be reconfigured. In this case, the affected agent may retain its contacts with corresponding agents without taking any action.

If the impact is not acceptable for an affected agent due to delay or extra cost incurred, it will apply CNP to determine the best services provided by the existing alternative downstream agents. If there exists alternative downstream agents that can provide the required service, the affected agent cancels contracts with existing downstream agents and establishes contracts with the new downstream agents. Otherwise, the affected agent will respond to its upstream and downstream affected agents to cancel the contacts. The upstream affected agent that received the request to cancel contract will apply CNP to attempt to find alternative downstream agents to reconfigure the workflow.

If the change detected by an affected agent is a structural change, the affected agent will apply CNP to attempt to find alternative downstream agents to reconfigure the workflow in execution. As structural changes include addition, modification and removal of some agents in a workflow system, the resulting configuration may add, replace and/or remove some of the agents in the existing workflow.

If the affected agent finds an alternative solution that will improve the current solution after applying CNP, it will replace the current solution with the alternative solution via cancellation of some of the existing contracts with the existing agents and establishment of new contracts with some new agents.

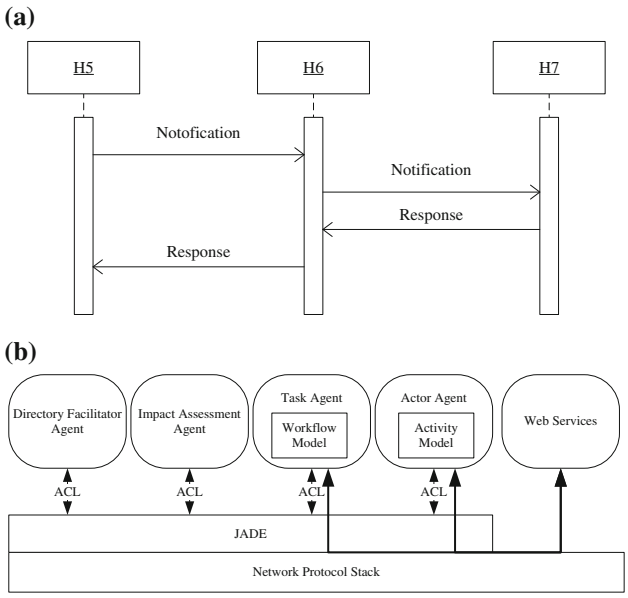

Fig. 7 **a** Notification of change and response. **b** Architecture of prototype system

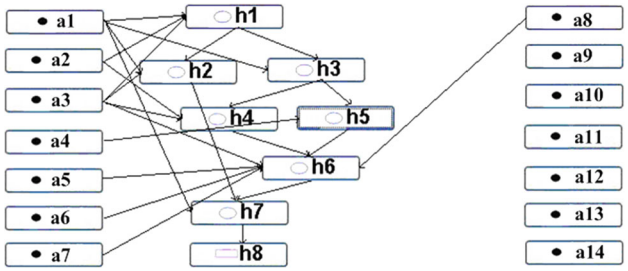

Fig. 8 Contracts established between holons

The notification message carries the impact information due to changes. To make each agent interprets the impact information consistently and interoperate with other, a XML schema is defined to represent the notification message and the impact information. Note that impact information only summarizes the impact of the change on the execution of workflow. It does not include the detailed workflow information.

Figure 7a illustrates how the notification messages and response messages propagate through the affected agents. In Fig. 7a, H_5 detects a change that indicates a significant delay in manufacturing. H_5 will evaluate the impact of the delay and notify H_6. On receiving the notification, H_6 will evaluate the impact of the delay and notify H_7 in turn. H_7 will evaluate the overall impact and then respond to H_6 based on the overall impact. The overall impact will also be forwarded to H_5 by H_6.

The aforementioned design methodology can be applied to attain self-adaptation of workflows in multi-agent systems based on JADE. Figure 7b shows a proposed architecture.

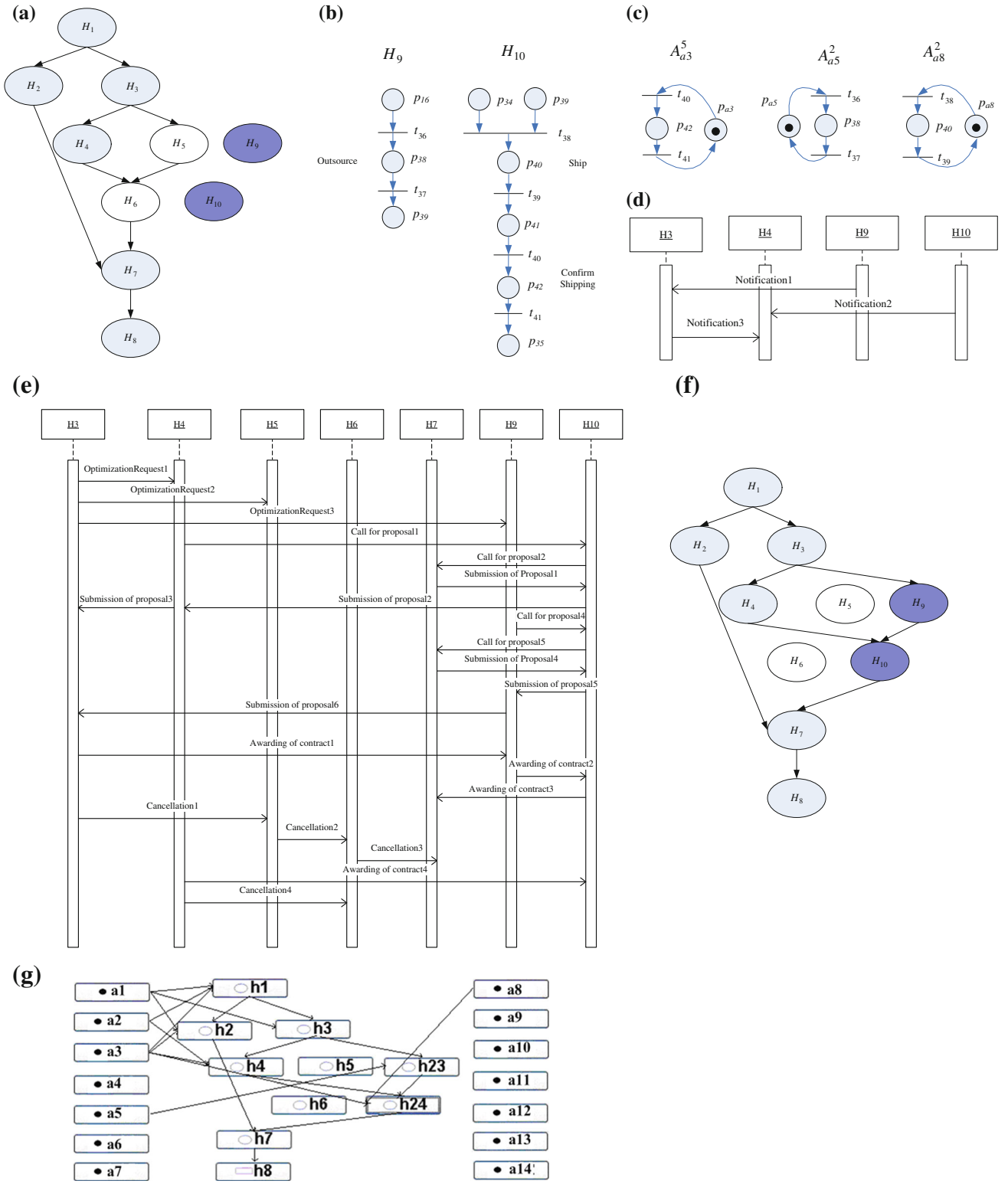

Fig. 9 **a** Two task agents added to the system. **b** Models of two task agents H_9 and H_{10} added to the system. **c** Models of three actor agents' activities added to the system. **d** Notification of new agents. **e** Interaction Diagram. **f** H_3 and H_4 have been replaced by H_5 and H_6 in adapted holarchy. **g** Adapted holarchy in the system

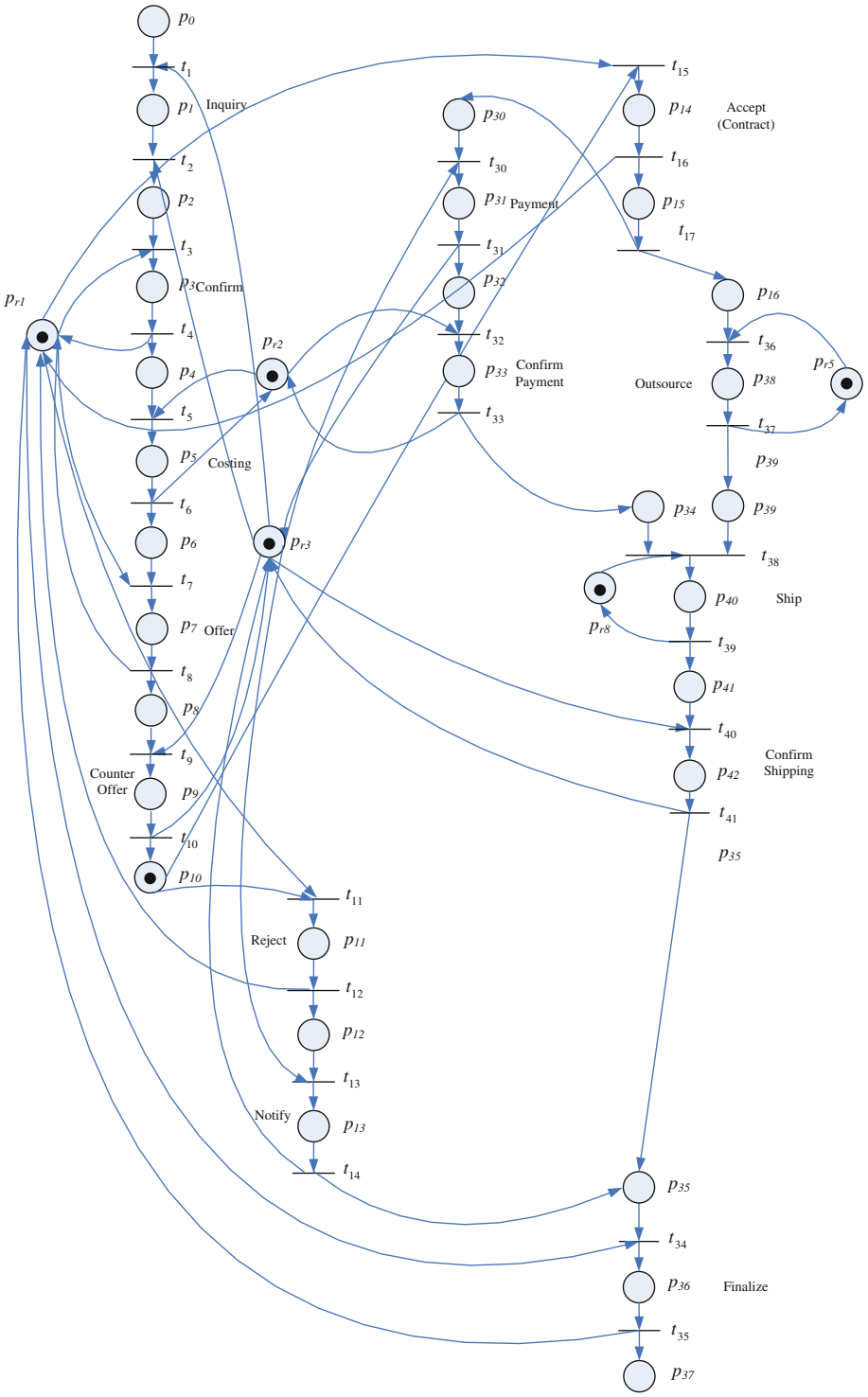

Fig. 10 Complete Petri net model for the adapted holarchy

The impact assessment agent implements the codes to compute the impact function based on the workflow model. It may be invoked by task agents to achieve self-adaptation of workflows. Parsing of PNML is implemented as Web Services that can be reused by agents to achieve interoperability.

Application scenario

Our prototype system consists of an editor tool to define agents in HMS. Figure 8 shows the twenty one agents defined for the example. Agents h_1–h_7 are task agents, h_8 is an order

agent and the remaining agents are actor agents. The different types of agents in HMS play different roles and therefore have different properties that need to be set and configured properly. After finishing the negotiation process, the contracts established between different agents are shown on the screen. Figure 8 shows the contracts established between task agents and actor agents.

In Fig. 9a, a holarchy is formed by H_1, H_2, H_3, H_4, H_5, H_6 and H_7. Suppose two task agents H_9 and $H_{10}$are added to the system to reflect the new outsource business process. This new business process virtually combines the operations of "manufacture", "quality assurance", shipping scheduling" and "packaging" into one operation called "outsource". Figure 9b shows the structure of the two task agentsH_9 and H_{10} added to the system. Figure 9c shows the models of three actor agents' activities added to the system. In this case, H_9 and H_{10} will notify the existing agents in the holarchy of their presence. Figure 9d illustrates how the notification messages are forwarded to H_3 and H_4 by H_9 and H_{10}. On receiving the notification, each agent will check whether any of its terminal output places is influenced by H_9 or H_{10}. For this example, only task agentsH_3 and H_4 satisfy this condition. In addition to the notification messages sent by the newly added agents, H_3 also notifies its downstream agent H_4 that it has received a notification to indicate that it will initiate an optimization process. In this example, agent H_3 applies CNP to attempt to find an alternative solution.

Figure 9e shows the interaction diagram of the reconfiguration process following the event that H_3 has received the notification indicating that H_9 and H_{10} have been added to the system. H_3 sends an optimization request to H_4, H_5, and H_9. As H_5 is not directly influenced by H_9 and H_{10}, it ignores the optimization request. H_4 and H_9 then apply CNP to optimize the processes.

The resulting downstream agents of H_3 that forms the solution for agent H_3 is H_3, H_4, H_9, H_{10} and H_7. Suppose the cost of this solution is lower than that of the solution H_3, H_4, H_5, H_6 and H_7. In this case, the new solution will replace H_5 and H_6 with H_9, H_{10} and, respectively. The resulting holarchy is shown in Fig. 9f. Figure 9g illustrates the resulting holarchy in our software system. Note that actor agents a_3, a_5 and a_8 also belong to the holarchy in Fig. 9g to perform the three actor agents' activities added to the system in Fig. 9c. Note that the contracts with actor agents a_4, a_6 and a_7 have been removed from Fig. 9e. Figure 10 shows the complete Petri net model for the adapted holarchy. An example of action list for actor agent is shown in Fig. 11 of Hsieh and Lin (2012).

Figure 10 shows the complete Petri net model of the adapted holarchy. Based on the complete Petri net model, we can generate context-aware action lists for actor agents. The details about how to develop context-aware workflow management system is out of the scope of this paper. Interested readers may refer to Hsieh and Lin (2012) for the details about how to develop a real context-aware workflow application.

Conclusion

Effective workflow management software systems must be able to accommodate changes by adapting to changing environments, resource variability and user needs to support the operations of modern enterprises. Self-adaptation of software systems relies on a flexible and reconfigurable architecture to reap the benefits of stability in the face of disturbances, flexibility in the face of change, and efficient use of available resources. HMS provides a flexible and reconfigurable architecture to deal with changes based on dynamic organization and collaboration of autonomous agents. An agent is an autonomous, co-operative and intelligent entity able to collaborate with other agents to process the tasks. In HMS, a set of agents that autonomously cooperate to achieve a goal forms a holarchy. The loosely coupled structure of agents in HMS makes it possible to attain for the benefits of self-adaptation in software systems. The objectives of this paper are to propose a viable software design methodology to achieve self-adaptation in workflow management systems based on HMS architecture.

To achieve the research objectives, we propose a novel approach to dynamically composing workflow services based on HMS architecture. We formulate a workflow adaptation problem (WAP) and propose an interaction mechanism based on contract net protocol (CNP) (Smith 1980; FIPA 2002b) to find a solution to WAP and compose the services based on HMS. The interaction mechanism relies on a service publication and discovery scheme to find a set of task agents and a set of actor agents to form a holarchy to compose the required services in HMS. We propose a self-adaptation scheme to reconfigure the holarchy as well as the composed services to respond to changes in processes.

Instead of modeling a service as a static passive component, we model a service as an agent. In our approach, the service provided by each agent is modeled as an atomic service. The atomic services can be classified into two categories: atomic tasks and atomic activities. An agent hosts an atomic service can be combined with the atomic services provided by other agents dynamically to compose the desired services and achieve the goal.

To facilitate composition of a desired service based on atomic services, a model is required to represent atomic services. Due to the well-established formal methods for modeling, analysis as well as verification and the availability of the PNML standardized interchange format to facilitate composition of services, Petri net is adopted in this paper to model atomic services. Our design methodology is broken down

into four parts. First, we construct task models and actor activity models in Petri nets using PNML editors. Second, we propose a service publication and discovery scheme based on Petri net models. Third, an interaction mechanism based on CNP is proposed to solve WAP. Fourth, we propose a self-adaptation scheme to re-structure the holarchy to reflect and respond to the changes based on cooperation of agents in HMS.

Finding a solution from scratch from the point a change occurs is not an appropriate approach as it may lead to chaos. An effective approach to self-adaptation must be based on the nominal solution so that the impact on the operation of existing agents can be significantly reduced. Whenever a change occurs, the agent that detects the changes in business processes and failures to perform an activity will notify the existing agents the changes detected. Each agent that receives the notification from other agents will assess the impact on itself due to the detected changes. An affected agent will apply CNP to determine the best services provided by the existing downstream agents. If the best services provided by the existing downstream agents remain intact, no reconfiguration is required. In this case, all new workflow instances will follow the old workflow routes. Otherwise, the affected agent will cancel contracts with existing downstream agents and establish new contracts with the best agents. All new workflow instances will follow the new workflow routes stemming from the affected agent due to the establishment of new contracts with the best agents. Existing running workflow instances that still not reach the affected agent also follow the new workflow routes. However, existing running workflow instances that have reached the affected agent will follow the old workflow routes to avoid chaos. We have proposed the aforementioned design methodology to facilitate the development of self-adaptation in multi-agent systems and illustrate by an application scenario.

This paper is differentiated from the results reported in van der Aalst and Basten (2002), van der Aalst and Jablonski (2000), Rinderle et al. (2004), Cicirelli et al. (2010) as it focuses on the development of autonomic reconfiguration scheme to deal with process changes based on HMS architecture. Rinderle et al. survey different approaches to deal with changes in workflows. They observe that process changes can take place at two levels—the process type and the process instance level. Instance-specific changes affect only single process instances. As opposed to this, a collection of related instances may have to be adapted to changes at the process type level. van der Aalst and Jablonski propose four approaches to achieving workflow evolution (van der Aalst and Jablonski 2000), including Forward recovery, Backward recovery, Proceed and Transfer. Recently, Cicirelli, Furfaro and Nigro propose an approach to decentralized enactment and dynamic reconfiguration of business processes based on a service-oriented architecture (Cicirelli et al. 2010).

This paper is different from the studies of Garlan et al. (2004), Kasten and McKinley (2004), Kon et al. (2005). The Rainbow framework proposed by Garlan et al. (2004) is a general architecture-based self-adaptation framework that uses software architectures and a reusable infrastructure to support self-adaptation of software systems. However, their approach lacks component composition support which is also important in building applications. The Perimorph framework proposed by Kasten and McKinley (2004) enables an application designer to quantify and verify changes to achieve runtime composition and state management for an adaptive system. However, due to lack of a clearly defined component model, it is hard to extend their approach to cross applications adaptation. Kon et al. (2005) propose an integrated architecture that supports automatic configuration and dynamic resource management for managing complex dependencies between components in distributed component-based systems. In contrast with the above mentioned methods, the advantages of our design methodology include specification of task models and actor activity models in PNML in analysis and design of efficient adaptation scheme. This paper is differentiated from Hsieh (2010) as our self-adaptation scheme is able to deal with both structural changes and non-structural changes in workflow systems whereas Hsieh (2010) can only accommodate changes in available resources.

Acknowledgments This paper is currently supported in part by National Science Council of Taiwan under Grant NSC102-2410-H-324-014-MY3.

References

Ambros-Ingerson, J., & Steel, S. (1988). Integrating planning, execution and monitoring. In *7th national conference on artificial intelligence (AAAI-88)*. St. Paul, USA.

ATHENA (2003). *Advanced technologies for interoperability of heterogeneous enterprise networks and their applications*. FP6-2002-IST1, Integrated project.

A.O. Software. (2012). *JACK intelligent agents manual*. http://www.agent-software.com.

B. Telecommunications. (2002). *Zeus*. http://more.btexact.com/projects/agents/zeus/index.htm.

Balasubramanian, S., Brennan, R. W., & Norrie, D. H. (2001). An architecture for metamorphic control of holonic manufacturing systems. *Computers in Industry*, *46*, 13–31.

Billington, J., Christensen, S., van Hee, K., Kindler, E., Kummer, O., Petrucci, L., et al. (2003). The Petri net markup language: concepts, technology and tools. *Lecture notes in computer science* (Vol. 2679, pp. 483–505).

Blanc, S., Ducq, Y., & Vallespir, B. (2007). Evolution management towards interoperable supply chains using performance measurement. *Computers in Industry*, *58*(7), 720–732.

Bratman, M. E., Israel, D. J., & Pollack, M. E. (1988). Plans and resourcebounded practical reasoning. *Computational Intelligence*, *4*, 349–355.

Bresciani, P., Giorgini, P., Giunchiglia, F., Mylopoulos, J., & Perini, A. (2004). TROPOS: An agent oriented software development

methodology. *Journal of Autonomous Agents and Multi-agent Systems*, *8*(3), 203–236.
C4ISR Architecture Framework. (1998). *Levels of information systems interoperability (LISI)*.
Chen, D., & Vernadat, F. (2002). Enterprise interoperability: A standardisation view. In K. Kosanke, et al. (Eds.), *Enterprise inter-and-intra organisational integration* (pp. 273–282). Dordrecht: Kluwer. ISBN:1-4020-7277-5.
Chen, D., Dassisti, M., & Tsalgatidou, A. (2005). *Interoperability knowledge corpus*. Deliverable DI.1. Workpackage DI, INTEROP NoE, November 25,2005.
Chen, D., & Vernadat, F. (2004). Standards on enterprise integration and engineering—a state of the art. *International Journal of Computer Integrated Manufacturing*, *17*(3), 235–253.
Chen, D., Doumeingts, G., & Vernadat, F. (2008). Architectures for enterprise integration and interoperability: Past, present and future. *Computers in Industry*, *59*, 647–659.
Chituc, C.-M., Toscano, C., & Azevedo, A. (2008). Interoperability in collaborative networks: Independent and industry-specific initiatives—the case of the footwear industry. *Computers in Industry*, *59*, 741–757.
Chituc, C.-M., Azevedo, A., & Toscano, C. (2009). A framework proposal for seamless interoperability in a collaborative networked environment. *Computers in Industry*, *60*(5), 317–338.
Christensen, J. (1994). Holonic manufacturing systems—initial architecture and standards directions. In *Proceedings of the first European conference on holonic manufacturing systems*, Hannover, Germany.
Cicirelli, F., Furfaro, A., & Nigro, L. (2010). A service-based architecture for dynamically reconfigurable workflows. *The Journal of Systems and Software*, *83*, 1148–1164.
Cossentino, M. (2005). From requirements to code with the PASSI methodology. In B. Henderson-Sellers & P. Giorgini (Eds.), *Agent-oriented methodologies* (pp. 79–106). Hershey, PA: Idea Group.
Coulson, G., Blair, G., Grace, P., Taiani, F., Joolia, A., Lee, K., et al. (2008). A generic component model for building systems software. *ACM Transactions on Computer Systems*, *26*(1), 1–42.
DeLoach, S. A., & Kumar, M. (2005). Multi-agent systems engineering: An overview and case study. In B. Henderson-Sellers & P. Giorgini (Eds.), *Agent-oriented methodologies* (pp. 236–276). Hershey, PA: IDEA Group Publishing.
Emorphia. (2003). *FIPA-OS*. http://fipa-os.sourceforge.net/index.htm.
Ferber, J., & Gutknecht, O. (1998). A meta-model for the analysis and design of organizations in multi-agent systems. In *3rd international conference on multi-agent systems (ICMAS'98)*, pp. 128–135. Paris, France.
Fikes, R. E., & Nilsson, N. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, *5*(2), 189–208.
FIPA. (2002a). *FIPA ACL message structure specification*. http://www.fipa.org/specs/fipa00061/SC00061G.pdf.
FIPA. (2002b). *FIPA interaction protocols specifications*. http://www.fipa.org/repository/ips.php3.
Garlan, D., Cheng, S. W., Huang, A. C., Schmerl, B., & Steenkiste, P. (2004). Rainbow: Architecture-based self-adaptation with reusable infrastructure. *Computer*, *37*(10), 46–49.
Hsieh, F. S. (2008). Holarchy formation and optimization in holonic manufacturing systems with contract net. *Automatica*, *44*(4), 959–970.
Hsieh, F. S. (2009). Dynamic composition of holonic processes to satisfy timing constraints with minimal costs. *Engineering Applications of Artificial Intelligence*, *22*(7), 1117–1126.
Hsieh, F. S. (2010). Design of reconfiguration mechanism for holonic manufacturing systems based on formal models. *Engineering Applications of Artificial Intelligence*, *23*(7), 1187–1199.
Hsieh, F. S., & Chiang, C. Y. (2011). Collaborative composition of processes in holonic manufacturing systems. *Computers in Industry*, *62*(1), 51–64.
Hsieh, F. S., & Lin, J. B. (2012). Context-aware workflow management for virtual enterprises based on coordination of agents. *Journal of Intelligent Manufacturing*. doi:10.1007/s10845-012-0688-8.
Huhns, M. N., & Singh, M. P. (2005). Service-oriented computing: Key concepts and principles. *IEEE Internet Computing*, *9*(1), 75–81.
IDEAS (2002). *Thematic network, IDEAS: Interoperability development for enterprise application and software–roadmaps*, Annex 1–DoW.
Iglesias, C. A., & Garijo, M. (2005). The agent-oriented methodology MAS-commonKADS. In B. Henderson-Sellers & P. Giorgini (Eds.), *Agent-oriented methodologies* (pp. 46–78). Hershey, PA: IDEA Group Publishing.
ISO 11354-1. (2011). *Advanced automation technologies and their applications—requirements for establishing manufacturing enterprise process interoperability—part 1: Framework for enterprise interoperability*.
ISO 18629-1. (2004). *Industrial automation systems and integration—process specification language—part 1: Overview and basic principles*.
ISO/IEC 15909-2. (2011). *Systems and software engineering—high-level Petri nets—part 2: Transfer format*.
Kasten, E. P., & McKinley, P. K. (2004). Perimorph: Run-time composition and state management for adaptive systems. In *24th international conference on distributed computing systems workshops, proceedings*, pp. 332–337.
Koestler, A. (1967). *The ghost in the machine*. London: Hutchinson.
Kon, F., Marques, J. R., Yamane, T., Campbell, R. H., & Mickunas, M. D. (2005). Design, implementation, and performance of an automatic configuration service for distributed component systems. *Software-Practice & Experience*, *35*(7), 667–703.
KQML Specification Document. (1993). *UMBC AgentWeb*. http://www.cs.umbc.edu/kqml/.
Leitão, P., & Restivo, F. (2006). ADACOR: A holonic architecture for agile and adaptive manufacturing control. *Computers in Industry*, *57*(2), 121–130.
Murata, T. (1989). Petri nets: Properties. analysis and applications. *Proceedings of the IEEE*, *77*(4), 541–580.
Naudet, Y., Latour, T., Guedria, W., & Chen, D. (2010). Towards a systemic formalisation of interoperability. *Computers in Industry*, *61*(2), 176–185.
OASIS. (2009). Web services business process execution language version 2.0. http://docs.oasis-open.org/wsbpel/2.0/OS/wsbpel-v2.0-OS.html.
Object Management Group, (2009). Business process modeling notation. http://www.bpmn.org.
Oreizy, P., Gorlick, M. M., Taylor, R. N., Heimbigner, D., Johnson, G., Medvidovic, N., et al. (1999). An architecture-based approach to self-adaptive software. *IEEE Intelligent Systems & Their Applications*, *14*(3), 54–62.
Ort, E. (2005). *Service-oriented architecture and web services: Concepts, technologies, and tools*. Available at http://java.sun.com/developer/technicalArticles/WebServices/soa2/index.html.
Padgham, L., & Winikoff, M. (2005). Prometheus: A practical agent-oriented methodology. In B. Henderson-Sellers & P. Giorgini (Eds.), *Agent-oriented methodologies* (pp. 107–135). Hershey, PA: IDEA Group Publishing.
Panetto, H., & Molina, A. (2008). Enterprise integration and interoperability in manufacturing systems: Trends and issues. *Computers in Industry*, *59*(7), 641–646.
Rinderle, S., Reichert, M., & Dadam, P. (2004). Correctness criteria for dynamic changes in workflow systems: A survey. *Data and Knowledge Engineering*, *50*(1), 9–34.

Smith, R. G. (1980). The contract net protocol: High-level communication and control in a istributed problem solver. *IEEE Transactions on Computers*, *29*(12), 1104–1113.

SOAP Specification. (2007). http://www.w3c.org/TR/SOAP.

Tran, Q.-N. N., & Low, G. (2008). MOBMAS: A methodology for ontology-based multi-agent systems development. *Information and Software Technology*, *50*, 697–722.

UDDI Specification. (2007). http://www.uddi.org.

Web Services Description Language (WSDL) Specification. (2001). http://www.w3c.org/TR/wsdl.

van der Aalst, W. M. P. (1998). The application of Petri nets to workflow management. *Journal of Circuits, Systems, and Computers*, *8*(1), 21–66.

van der Aalst, W. M. P., & Basten, T. (2002). Inheritance of workflows: An approach to tackling problems related to change. *Theoretical Computer Science*, *270*(1), 125–203.

van der Aalst, W. M. P., & Jablonski, S. (2000). Dealing with workflow change: Identification of issues and solutions. *International Journal of Computer Systems Science and Engineering*, *15*(5), 267–276.

Weber, M., & Kindler, E. (2002). The Petri net markup language. http://www2.informatik.hu-berlin.de/top/pnml/download/about/PNML_LNCS.pdf.

Weichhart, G., Feiner, T., & Stary, C. (2010). Implementing organizational interoperability—the SUddEN approach. *Computers in Industry*, *61*(2), 152–160.

Wood, S. (1993). *Planning and decision making in dynamic domains*. Chichester: Ellis Horwood.

Wooldridge, M. (1999). Intelligent agents. In G. Weiss (Ed.), *Multiagent systems: A modern approach to distributed artificial intelligence* (pp. 27–77). London: The MIT Press.

Wooldridge, M., Jennings, N. R., & Zambonelli, F. (2005). Multi-agent systems as computational organizations: The Gaia methodology. In B. Henderson-Sellers & P. Giorgini (Eds.), *Agent-oriented methodologies* (pp. 136–171). Hershey, PA: IDEA Group Publishing.

Workflow Management Coalition. (1998). http://www.wfmc.org.

Workflow Management Coalition. (1999). *The workflow management coalition specifications: Terminology and glossary*. http://www.wfmc.org.

Wyns, J. (1999). *Reference architecture for holonic manufacturing*. Ph.D. dissertation, PMA Division, Katholieke Universiteit, Leuven, Belgium.